NATIONAL UNIVERSITY OF MODERN LANGUAGES

Assignment # 01

SUBMITTED BY:

ZULQARNAIN HASSAN

SUBMITTED TO:

SIR M WAQAR

Subject: …….. data structure & algorithm

Roll No: …… 21379

Department: ……. Software engineering

Section: ……. BSSE-III-A-afternoon

Circular link list:

The circular linked list is a linked list where all node are connected to form a circle. In a circular linked list, the first node and last node are connected to each other, forming a circle. There is no NULL at the end.

There are generally two types of circular linked lists:

- Circular singly linked list:

  In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We traverse the circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning or end. No null value is present in the next part of any of the nodes.

Example

```
#include<iostream>

using namespace std;

struct node{

   int data;

   node* next;

};

node* creat(int);

void insert(node*);

void show();

node* start=NULL;

main()

{

   int n1,n2,n3,n4;

   cout<<"enter four numbers:   \n";

   cin>>n1>>n2>>n3>>n4;

   node* ptr1=creat(n1);

   node* ptr2=creat(n2);

   node* ptr3=creat(n3);

   node* ptr4=creat(n4);
```

```
    insert(ptr1);
    insert(ptr2);
    insert(ptr3);
    insert(ptr4);
  // show();
   ptr4->next=ptr1;
   show();
}
node* creat(int n){
   node* ptr=new node;
   ptr->data=n;
   ptr->next=NULL;
   return ptr;
}
void insert(node* ptr)
{
   node* tem=start;
   if(start==NULL)
   {
      start=ptr;
   }
   else
   {
     while(tem->next!=NULL)
     {
       tem=tem->next;
     }
     tem->next=ptr;
   }
```

```
}

void show()

{

   cout<<"output";

   node* tem=start;

   while(tem!=NULL)

   {

      cout<<tem->data<<endl;

      tem=tem->next;

      if(tem==start)

      break;

   }

}
```

```
PS E:\C++> cd "e:\C++\" ; if ($?) { g++ tut4.cpp -o tut4 } ; if ($?) { .\tut4 }
enter four numbers:
1
2
3
4
output1
2
3
4
PS E:\C++>
```

- Circular Doubly linked list:

  Circular Doubly Linked List has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by the previous and next pointer and the last node points to the first node by the next pointer and also the first node points to the last node by the previous pointer.

Example

```
#include<iostream>

using namespace std;

struct node{

   int data;

   node* next;
```

```
    node* prev;
};
node* creat(int);
void insert(node*);
void show();
node* start=NULL;
main()
{
    int n1,n2,n3,n4;
    cout<<"enter four numbers:   \n";
    cin>>n1>>n2>>n3>>n4;
    node* ptr1=creat(n1);
    node* ptr2=creat(n2);
    node* ptr3=creat(n3);
    node* ptr4=creat(n4);
     insert(ptr1);
     insert(ptr2);
     insert(ptr3);
     insert(ptr4);
   // show();
    ptr4->next=ptr1;
    ptr1->prev=ptr4;
    show();
}
node* creat(int n){
    node* ptr=new node;
    ptr->data=n;
    ptr->prev=NULL;
    ptr->next=NULL;
```

```
    return ptr;
}
void insert(node* ptr)
{
    node* tem=start;
    if(start==NULL)
    {
       start=ptr;
    }
    else
    {
     while(tem->next!=NULL)
     {
       tem=tem->next;
     }
     tem->next=ptr;
     ptr->prev=tem;
    }
}
void show()
{
    cout<<"output\n";
    node* tem=start;
    while(tem!=NULL)
    {
       cout<<tem->data<<endl;
       tem=tem->next;
       if(tem==start)
       break;
```

```
    }

}
```

```
PS E:\C++> cd "e:\C++\" ; if ($?) { g++ tut5.cpp -o tut5 } ; if ($?) { .\tut5 }
enter four numbers:
1
2
3
4
output
1
2
3
4
PS E:\C++>
```

Advantages of Circular Linked Lists:

- Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.
- Useful for implementation of a queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use a circular linked list. We can maintain a pointer to the last inserted node and the front can always be obtained as next of last.
- Circular lists are useful in applications to repeatedly go around the list.
- Circular Doubly Linked Lists are used for the implementation of advanced data structures like the Fibonacci Heap.
- Implementing a circular linked list can be relatively easy compared to other more complex data structures like trees or graphs.

Disadvantages of circular linked list:

- Compared to singly linked lists, circular lists are more complex.
- Reversing a circular list is more complicated than singly or doubly reversing a circular list.
- It is possible for the code to go into an infinite loop if it is not handled carefully.
- It is harder to find the end of the list and control the loop.
- Although circular linked lists can be efficient in certain applications, their performance can be slower than other data structures in certain cases, such as when the list needs to be sorted or searched.
- Circular linked lists don't provide direct access to individual nodes